UNIVERSITY OF MONS

FACULTY OF SCIENCES

Comparison of the three protocols: SAML, OpenID Connect, and OAuth 2.0 for business

Tonesko DJOUFO

Master's degree in Computer Science

Tonesko.DjoufoTafejo@student.umons.ac.be

School year 2025-2026

Thesis completed in the **networks and telecommunications,** as part of **Master's degree in Computer Sciences, specialized purpose.**

Director : Alain BUYS

UMONS Université de Mons

Faculté des Sciences

The author, DJOUFO TAFEJO Tonesko, certifies that he has complied with the ethical rules in force, including the University's charter on the use of Artificial Intelligence.

Acknowledgments

Contents

Introduction **5**

1 Security protocol fundamentals **6**
1.1 Key concept 6
1.1.1 Authentication, authorisation and federation 6
1.1.2 Tokens, assertion and session 7

2 Deuxième chapitre **10**

3 Conclusion **11**

Bibliographie **11**

Annexe A: Ce que je ne veux pas mettre dans la partie principale du document **13**

Annexe B: Une autre pour la route **13**

Introduction

The different parts of this work are discussed in the next chapters.

Chapter 1

Security protocol fundamentals

1.1 Key concept

1.1.1 Authentication, authorisation and federation

To understand the differences between authentication, authorisation, and federation, it is important to first define each concept and clarify their respective roles in the context of enterprise identity and access management.

Authentification

Authentication is the process by which an application confirms the identity of a user, device, or system. It is a critical component of security, as it ensures that only legitimate entities can access sensitive information or perform certain actions. Authentication can be achieved through various methods, including passwords, biometrics (such as fingerprint or facial recognition), security tokens, or multi-factor authentication (MFA), which combines multiple methods for enhanced security. The primary goal of authentication is to establish trust and verify the identity of users or systems before granting access to resources or services [3].

For example, when a student logs into their university portal, they are required to provide their matriculation number, password, and sometimes a second factor such as a code sent to their mobile device to confirm their identity based on the credentials they registered with.

Authorisation

Authorisation, by contrast, is the process of granting a user or system access to a resource on behalf of another user or system [3]. A critical distinction must be drawn here: authorisation does not necessarily imply that the granting system knows who is making the request — only that a valid authorisation credential (such as a token or code) has been presented. As long as this credential is valid, access is granted, regardless of the actual identity of the requester.

Consider a user who wishes to publish a photo on a social media platform: the platform requests access to the user's photo gallery, not by asking for the user's credentials, but by obtaining a limited, scoped permission to access only that specific resource. This separation of identity from access rights is foundational to modern authorisation frameworks, and forms the core purpose of OAuth 2.0, as will be examined in Chapter 2.

Federation

Federation is also known as identity federation. It allows users to access multiple services or applications by using a single, agreed-upon set of credentials. This means that users can authenticate once and gain access to resources across different domains or organizations without needing to log in separately for each service [4].

Federation is typically implemented through some protocols such as SAML (Security Assertion Markup Language) [4] or OpenID Connect [3], which standardise the exchange of authentication and authorisation data between an Identity Provider (IdP), a trusted authority that holds user credentials, and a Service Provider (SP) that uses identity assertions.

For example, a student may use their university credentials to access different online services provided by the university, such as email, learning management systems, video-conference applications, and library ressources, without needing to have separate accounts on each platform. Federation simplifies the user experience by reducing the number of credentials users need to remember and manage, while also enhancing security by centralizing authentication and access control.

Summary

To summarise, authentication care about the identity of the user ("Who you are?"), while authorisation focus on the permissions and access rights ("What you can do?"), and federation is about enabling seamless access across different services ("How can your identity be trusted across different systems?"). These concepts are deeply interrelated: a system may handle authorisation without authentification (OAuth 2.0), authentication without federation, or combine all three within a unified framework (OpenID Connect, SAML). The way each of the three protocols OAuth 2.0, OIDC, and SAML handles these three dimensions largely determines their respective strengths, weaknesses, and domains of applicability in an enterprise context.

A useful analogy to understand this relationship is to think of a key (authentication) that grants entry to a building, where different rooms may have different access levels (authorisation). Federation acts as a master key that grants access to multiple buildings (apllications or services).

1.1.2 Tokens, assertion and session

In the context of identity and access management protocols, tokens, assertions, and sessions are fundamental concepts by which authentification and authorisation are achieved and maintained. While these terms are sometimes used interchangeably, they represent distinct mechanisms, each with specific characteristics depending on the protocol in use.

Tokens

A token a piece of data that represents a credential about a user, system or an authorisation grant. Tokens serve as portable proof that can be presented to access protected resources without requiring the user to re-authenticate each time. As Richer and Sanso observe: "OAuth 2.0 is ultimately about tokens" [3]. A principle that extends beyond OAuth 2.0 to encompass OIDC and in a broader sense, SAML. Tokens can take various forms depending on the protocol and use case:

- **JSON Web Tokens (JWT):** These tokens are compact, self-contained, meaning they carry all necessary information within the token itself, for stateless validation. They are the standard format for OIDC ID Tokens and are increasingly used for OAuth 2.0 access tokens.

- **Opaque Tokens:** These tokens do not contain any information that can be directly interpreted by the client. They serves as references to server-side stored information. They require the recipient to query the emitting server (via token introspection, RFC 7662) to retrieve associated metadata. Opaque tokens offer better revocation control but at the cost of additional round-trips to the server for validation.
- **SAML Assertions:** XML-based statements that contain structured informations about the user's identity and privileges. While not commonly referred to as "token" in SAML terminology, assertions functionally serve the same purpose: they are signed to ensure integrity and authenticity, and are issued by the IdP and consumed by the SP to make access control decisions.

The fundamental distinction between these formats lies not in their purpose, but in their structure (XML vs JSON), size(the maximum supported size for SAML response is 128KB [1], while JWT can be much smaller $\approx$ 1KB usually), and validation mechanism (XML signatures for SAML assertions vs cryptographic signatures for JWTs).

In OAuth 2.0 and OIDC, multiple token types coexist within a single flow:

- **Access Tokens:** These tokens are used to access protected resources. They are short-lived and may be opaque or JWTs, depending on the implementation.
- **Refresh Tokens:** These tokens are used to obtain new access tokens without requiring the user to re-authenticate. They are long-lived and must be securely stored by the client. They enable "silent authentification" [4].
- **ID Tokens:** Specific to OIDC, these tokens contain information about the authenticated user and are always formatted as JWTs. They are consumed by the client application to establish a session or personalize the user experience.

In SAML, the assertion itself acts as both identity proof and authorisation credential, consolidationg these roles into a single XML structure.

Assertions

An assertion is a formal statement made by a trusted authority (IdP) about a subject (user or system). In the SAML protocol, assertions are the primary mechanism for transmitting authentication and authorisation information between the IdP and SP.

A SAML assertion is always digitally signed (using XML Signature) to ensure integrity and authenticity, and may optionally be encrypted for confidentiality. It contains field like (subject, authentication statement, attribute statement, conditions) that provide detailed information about the authentication event and the user's attributes. The assertion is issued by the IdP after successful authentication and is consumed by the SP to make access control decisions.

Sessions

A session represents a temporary, stateful interaction between a user and a system. Sessions are critical for maintaining context across multiple requests without requiring repeated authentification, thereby improving user experience while managing security trade-offs. It can be managed by the application itself (application-level session) or by the identity provider (IdP/OP session) in the context of federated authentication.

Moreover, session lifetimes vary significantly depending on the sensitivity of the application. For example, for an online banking application, it may last only few minutes, with mandatory re-authentification after inactivity. And for standard web applications, it lasts for hours or even days balancing depending on the develloper needs.

Beside that, proper session termination is critical for security. In SAML, the IdP can initiate Single Logout (SLO) to terminate sessions across all SPs sharing the same IdP session. Sometimes, it can also be a disablement of an user account at the IdP level by an administrator and the SP will be notified at the next authentication attempt [4]. For example, when a student graduates or leaves the university, their account is disabled. In OIDC, it exists different flow to handle session termination: the Relying Party (RP) Initiated Logout flow[1], Front Channel Logout[2], and Back Channel Logout[3] [2]. In OAuth 2.0, since it is primarily an authorisation protocol, session management is typically handled at the application level with token revocation(RFC 7009) mechanisms to invalidate sessions. This can lead to security risks if not properly implemented, as tokens may remain valid even after a user has logged out.

Comparative summary

Concept	**SAML**	**OpenID Connect**	**OAuth 2.0**
Token/Assertion	SAML Assertion	JSON Web Token (JWT)	Access, Refresh Tokens
Token LifeTime	Short(minutes)	Access: short, ID: short, Refresh: long	Access: short, Refresh: long
Session Management	Native(SSO, SLO)	Native(RP-initiated Logout,Front/Back Channel Logout)	None(delegated to application)
Signature Mechanism	XML signature	JWT signature	JWT signature

While OIDC and OAuth 2.0 does not differ enough in the token format, lifetime or signature mechanism, SAML differ in all these aspects.

[1] It occurs when a logout request originates from a RP than the IP

[2] It uses the user's browser to propagate logout requests to all relying parties

[3] It uses direct server-to-server communication between the identity provider and each relying party

Chapter 2

Deuxième chapitre

Un guide de rédaction scientifique en informatique est disponible sous la forme de deux documents sur moodle [1] (une version en français et une version adaptée en anglais). C'est de ces documents que sont extraites les informations concernant LaTeX du chapitre 1. C'est également ces documents qui ont servi de point de départ pour ce canevas, certains packages LaTeX pouvant dès lors sembler superflus. Qui peut le plus peut le moins ...

[1] français : `https://moodle.umons.ac.be/mod/resource/view.php?id=199575`
anglais : `https://orbi.umons.ac.be/handle/20.500.12907/8103`

Chapter 3

Conclusion

Bibliography

[1] AMAZON WEB SERVICES. Single sign-on — SAML 2.0-based federated authentication — in Client VPN. https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/federated-authentication.html, 2024.

[2] IBM CORPORATION. Openid connect (OIDC) single logout. https://docs.verify.ibm.com/ibm-security-verify-access/docs/oauth2-slo, 2024.

[3] RICHER, J., AND SANSO, A. *OAuth 2 in Action*. Manning Publications, Shelter Island, NY, 2017.

[4] WILSON, Y., AND HINGNIKAR, A. *Solving Identity Management in Modern Applications: Demystifying OAuth 2, OpenID Connect, and SAML 2*. Apress, San Francisco, CA, 2022.

Annexe A: Ce que je ne veux pas mettre dans la partie principale du document

Parce que c'est trop technique ...

Annexe B: Une autre pour la route

Pour faire joli ...